

模块四: 数据工程 (上)

第十五讲: 非结构化文本处理——用 LLM 构建论文速读助手

[解释]

第十五讲概览

本讲是数据工程模块的第三讲, 引入"非结构化文本"这一全新维度。实战场景是: 从 arXiv 论文预印本平台获取最新 AI 论文列表, 调用硅基流动的 LLM API 对每篇论文做相关性分类和中文摘要生成, 最终输出结构化的速读清单。

核心代码逻辑参考教师的 AIIA-Agent 项目中的 arXiv 论文推送流水线, 简化为纯 Python 教学版本。

一、课程承接：从结构化采集到非结构化理解

数据的边界在哪里？

[解释]

第一章：课程承接

本章确立 L13→L14→L15 的技能递进关系：从"处理现成数据"到"自动采集数据"再到"理解非结构化文本"。同时引入本讲要解决的真实痛点——科研文献信息过载。

本章包含 3 页内容：三讲的递进关系、痛点引入、本讲任务地图。

从结构化到非结构化：三讲的递进关系

	L13 (EDA)	L14 (爬虫)	L15 (LLM)
数据类型	数字、分类字段	HTML 表格	自然语言文本
数据来源	Kaggle CSV	网页爬取	arXiv RSS API
处理工具	Polars	requests + BS4	feedparser + LLM API
核心操作	统计、分组、过滤	解析、提取、保存	分类、翻译、摘要
输出	VIP 转化率分析	排名交叉对比	论文速读清单

💡 前两讲处理的是**可计算**的数据（数字、排名）。本讲处理的是**需要理解**的数据（自然语言）。这正是 LLM 的用武之地。

[解释]

结构化数据 vs 非结构化数据（文本维度）

L14 讲过结构化数据和非结构化数据的区别，但那时讲的是"HTML 标签包裹的表格"。本讲的非结构化数据是另一个层面：**语义层面的非结构化**。英文论文摘要虽然是纯文本，但其含义（研究方向、核心贡献、结论）需要语言理解能力才能提取。传统的正则表达式和关键词搜索无法准确完成这项任务。

痛点：论文看不过来

教师的困境

- arXiv 每天新增 500+ 篇 AI 领域论文
- 全英文，每篇摘要 200-300 词
- 需要快速判断：哪些和我的研究方向相关？
- 手动浏览一遍至少需要 2-3 小时

学生的困境

- 导师说"去 arXiv 上跟进最新进展"
- 打开一看：全是英文、密密麻麻
- 不知道哪些值得读、哪些可以跳过
- 语言障碍 + 信息过载 = 放弃

本讲目标：用 AI 帮我们做两件事——**分类**（哪些论文和我相关？）和**翻译摘要**（这篇论文到底在说什么？）。最终产出：一份中文的"本周必读"清单。

[解释]

arXiv 概述

arXiv（读作 "archive"）是由康奈尔大学运营的开放获取论文预印本平台，覆盖物理、数学、计算机科学等领域。其计算机科学分类下的 cs.AI（人工智能）、cs.CL（计算语言学）、cs.CV（计算机视觉）等子类每天会收到大量新投稿。

arXiv 提供免费的 RSS feed，格式为标准的 XML，包含论文标题、作者、摘要和链接。这使得程序化访问变得非常简单。

本讲任务地图

Phase 1: 获取论文列表

- 数据来源: arXiv RSS feed
- 工具: `feedparser` 库
- 输出: 20 篇最新论文 (标题 + 英文摘要)

Phase 2: LLM 分类

- API 提供商: 硅基流动 (SiliconFlow)
- 模型: DeepSeek-V3.2
- 输出: 每篇论文的相关性评分 + 理由

Phase 3: LLM 中文摘要

- 对高相关论文生成中文标题 + 100 字摘要
- JSON 结构化输出

Phase 4: 生成速读清单

- 合并分类和摘要结果
- 输出 DataFrame + CSV
- 按相关性评分排序

与 L14 的对比: L14 的四阶段是"采集→清洗→合并→分析"。L15 的四阶段是"获取→分类→摘要→清单"——结构相同, 但处理对象从"表格"变成了"文本"。

[解释]

流水线架构

本讲的数据流水线可以表示为: `arXiv RSS` → `feedparser` 解析 → LLM 分类评分 → LLM 中文摘要 → DataFrame → CSV。每个环节的输入是上一环节的输出, 形成完整的链路。

这个架构参考了 AIIA-Agent 项目的 `hot-articles-push.js` 脚本, 但简化了数据库缓存和推送环节, 仅保留核心的获取→处理→输出逻辑。

二、arXiv RSS：论文的“实时快递站”

不需要爬虫——arXiv 有官方 API

[解释]

第二章：arXiv RSS

本章介绍一种与 L14 爬虫不同的数据获取方式：通过网站提供的 RSS API 直接获取结构化数据。arXiv 的 RSS feed 返回标准 XML 格式，包含论文标题、作者、摘要和链接，无需解析复杂的 HTML 页面。

本章包含 3 页内容：RSS 基本概念、arXiv RSS URL 构造规则、feedparser 库的使用方法。

RSS 与爬虫的区别

L14 的方式：爬虫

- 目标是普通网页（HTML）
- 需要解析复杂的 HTML 标签
- 容易被网站拒绝（403）
- 页面结构变化时代码就会失效

L15 的方式：RSS API

- 目标是网站提供的接口
- 返回标准 XML 格式，结构固定
- 官方许可，不会被封禁
- 格式稳定，几乎不会变

💡 **优先用 API，不到万不得已不用爬虫。** 如果一个网站提供了 RSS、API 或数据下载，直接用官方渠道获取数据；只有在没有官方渠道时才考虑编写 HTML 爬虫。

[解释]

RSS (Really Simple Syndication)

RSS 是一种内容聚合标准，允许网站以 XML 格式发布结构化的内容摘要。用户（或程序）可以订阅 RSS feed，自动获取网站的最新更新，而无需手动访问网页。
在数据工程中，RSS 是比爬虫更优的数据获取方式，因为它是网站方主动提供的、格式稳定的数据接口，不会随页面改版而失效，也不存在被封禁的风险。

arXiv RSS：URL 构造规则

arXiv 为每个学科分类提供独立的 RSS feed。

URL 格式

`https://rss.arxiv.org/rss/{分类代码}`

常用分类代码

代码	领域
cs.AI	人工智能
cs.CL	计算语言学（NLP）
cs.CV	计算机视觉
cs.IR	信息检索
cs.LG	机器学习

返回内容（XML）

```
<item>
  <title>Attention Is All You Need</title>
  <link>http://arxiv.org/abs/2401.xxxx</link>
  <description>
    We propose a new architecture...
  </description>
  <dc:creator>Vaswani et al.</dc:creator>
  <category>cs.CL</category>
</item>
```

- `title`：论文标题
- `description`：英文摘要
- `link`：论文页面链接
- `category`：所属分类

[解释]

arXiv 分类体系

arXiv 的计算机科学分支（cs.*）下有约 40 个子分类。每个子分类有独立的 RSS feed，每天更新一次（工作日）。每次更新包含该分类当天新提交的所有论文。

一个分类的 RSS feed 通常包含 20-100 篇论文，每篇包含完整的标题、摘要（约 200-300 词）和元数据。这为我们的 LLM 处理提供了充足的输入数据。

feedparser: 一行代码获取论文

核心代码

```
import feedparser

# 获取 cs.AI 最新论文
url = "https://rss.arxiv.org/rss/cs.AI"
feed = feedparser.parse(url)

# 查看论文数量
print(f"获取到 {len(feed.entries)} 篇论文")

# 提取第一篇论文
paper = feed.entries[0]
print(f"标题: {paper.title}")
print(f"链接: {paper.link}")
print(f"摘要: {paper.summary[:100]}...")
```

与 L14 的对比

	L14 (HTML 爬虫)	L15 (RSS)
依赖库	requests + BS4	feedparser
代码行数	~20 行	3 行
解析复杂度	需要定位标签	自动解析
稳定性	页面结构变则失效	格式固定

 feedparser 自动处理了 XML 解析。每篇论文变成一个字典对象，直接用 `.title`、`.summary` 访问。

[解释]

feedparser 库

feedparser 是 Python 中最常用的 RSS/Atom 解析库。它能自动处理各种 RSS 版本 (RSS 1.0/2.0、Atom) 的差异，将 feed 内容统一转化为 Python 字典。

`feed.entries` 是一个列表，每个元素代表一篇论文。每个 entry 对象包含 `title` (标题)、`summary` (摘要)、`link` (链接)、`published` (发布时间) 等属性。

三、LLM API：让 AI 帮你读论文

从"人工阅读"到"人机协同阅读"

[解释]

第三章：LLM API 调用

本章是全讲的技术核心。介绍如何通过 OpenAI 兼容的 SDK 调用硅基流动（SiliconFlow）平台上的 LLM，实现论文分类和中文摘要生成。

本章包含 5 页内容：API 基本概念、硅基流动配置、OpenAI SDK 调用、分类 Prompt 设计、摘要 Prompt 设计。

API 基本概念

什么是 API?

API (Application Programming Interface) 是程序之间的通信接口。

用例对比:

- **网页版 ChatGPT:** 人类手动输入问题, 手动复制答案
- **API 调用:** 程序自动发送请求, 自动接收响应

API 调用的基本要素

要素	说明
Base URL	API 服务器地址
API Key	身份认证凭证 (相当于密码)
Model	使用哪个模型
Prompt	发送给 AI 的指令
Response	AI 返回的结果

为什么用 API 而不是网页版?

- **批量化:** 1 分钟处理 100 篇论文
- **自动化:** 可以定时执行
- **结构化输出:** 拿到 JSON, 直接变 DataFrame

[解释]

API 的技术原理

API 调用的本质是 HTTP 请求。客户端 (你的 Python 脚本) 向服务端 (LLM 提供商的服务器) 发送一个 POST 请求, 请求体包含模型名、Prompt 和参数。服务端处理后返回一个 JSON 响应, 其中包含 AI 生成的文本。OpenAI 定义了一套标准的 Chat Completions API 格式 (/v1/chat/completions), 许多国内外的 LLM 提供商 (包括硅基流动) 都兼容这一格式, 因此同一份代码可以在不同平台之间切换。

硅基流动（SiliconFlow）配置

平台简介

- 国内 LLM API 聚合平台
- 兼容 OpenAI 格式
- 支持 DeepSeek、Qwen 等主流模型

注册方式

1. 先看学习群内是否有同学发的邀请链接
2. 如果没有，使用教师链接注册：

`cloud.siliconflow.cn/i/uvQthMIy`

3. 通过邀请链接注册可获得 **14 元额度**

4. 注册后立即到

`cloud.siliconflow.cn/me/campaigns/inviter`

获取自己的邀请链接，**分享到学习群**

配置参数

参数	值
Base URL	<code>https://api.siliconflow.cn/v1</code>
模型	<code>deepseek-ai/DeepSeek-V3.2</code>
SDK	<code>openai</code> Python SDK

连接代码

```
from openai import OpenAI

client = OpenAI(
    api_key="你的 API Key",
    base_url="https://api.siliconflow.cn/v1"
)

# 测试连接
resp = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3.2",
    messages=[
        {"role": "user", "content": "你好"}
    ]
)
print(resp.choices[0].message.content)
```

⚠ API Key 安全规则：严禁在代码中硬编码 Key。使用环境变量：

```
api_key=os.getenv("SILICONFLOW_API_KEY")
```

[解释]

OpenAI 兼容格式

OpenAI 的 Chat Completions API 已成为 LLM 调用的行业标准。其核心参数包括：`model`（模型名）、`messages`（对话历史，包含 role 和 content）、`temperature`（随机性控制）、`response_format`（输出格式）。

硅基流动、月之暗面、智谱等国内平台均兼容这一格式，只需更换 `base_url` 即可切换平台，代码逻辑完全不需要修改。这就是“标准化接口”的工程价值。

分类 Prompt：判断论文相关性

任务定义

给定一个研究方向和一组关键词，让 LLM 对每篇论文打分：

- 0-0.4：无关
- 0.5-0.7：边缘相关
- 0.8-1.0：高度相关

返回格式

```
[
  {
    "index": 0, "score": 0.85,
    "reason": "提出新的注意力机制",
  },
  {
    "index": 1, "score": 0.3,
    "reason": "与主题无关"
  }
]
```

Prompt 模板

```
prompt = f"""你是一个 AI 研究论文分类助手。
请对以下论文判断其与主题
「{topic_name}」的相关性。
```

```
主题关键词：{keywords}
```

```
论文列表：
{papers_text}
```

```
请返回 JSON 数组，每个元素包含：
```

- index：论文编号
- score：相关性评分 0-1
- reason：一句话说明理由（中文）

```
仅返回 score >= 0.5 的论文。"""
```

💡 这个 Prompt 的结构和 L13 的 EDA Prompt 一样：角色 + 任务 + 数据 + 输出格式。唯一的新要素是要求返回 JSON。

[解释]

分类 Prompt 的设计要点

这个 Prompt 模板参考了 AIIA-Agent 项目中 `classifyArticles()` 函数的实现（`hot-articles-push.js` L166-180）。核心设计要点：

1. **明确的角色设定**：告诉 LLM 它是“论文分类助手”
2. **量化评分标准**：用 0-1 分值替代“相关/不相关”的二元判断
3. **结构化输出**：要求返回 JSON 数组，便于程序解析
4. **过滤阈值**：只返回 $\text{score} \geq 0.5$ 的论文，减少返回数据量

摘要 Prompt：生成中文速读版

任务定义

对已通过分类筛选的论文，生成：

- 中文标题
- 100-150 字的中文摘要

质量要求

1. 摘要必须覆盖：研究问题、方法、结论
2. 保留英文缩写（LLM、RAG 等）
3. 符合中文学术风格

Prompt 模板

```
prompt = f"""你是专业的 AI 学术论文翻译助手。

论文标题: {title}
英文摘要: {abstract}

## 规则
1. 中文摘要 100-150 字，提炼核心贡献
2. 必须包含：研究问题、提出方法、主要结果
3. 保留英文缩写（LLM、RAG 等），
   括号附注中文
4. 同时翻译标题

返回 JSON:
{"title_zh": "中文标题",
 "summary_zh": "100-150字摘要"}"""
```

[解释]

摘要 Prompt 的设计要点

这个模板参考了 AIIA-Agent 的 `generateSummaries()` 函数（`hot-articles-push.js` L246-260）。关键设计决策：

1. **字数约束**：100-150 字是经过实践验证的最佳长度——足够覆盖要点，又不会太冗长
2. **三要素强制覆盖**：研究问题+方法+结论，确保摘要要有实质内容
3. **术语保留策略**：英文缩写直接保留（如“RAG”），较长的术语用括号附中文（如“Retrieval-Augmented Generation（检索增强生成）”）
4. **JSON 输出**：便于后续合并到 DataFrame

四、批量流水线：从 1 篇到 N 篇

工程化思维："先打样再量产"

[解释]

第四章：批量流水线

本章将 Prompt 从"单篇测试"扩展为"批量处理"，并引入 JSON 解析、错误处理、成本核算等工程化环节。
本章包含 4 页内容：单篇测试流程、批量循环处理、结构化输出转 DataFrame、成本核算与优化。

单篇测试：先调通 1 篇

代码

```
import json

# 取第一篇论文
paper = feed.entries[0]

# 调用 LLM 做分类
resp = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3.2",
    messages=[{
        "role": "user",
        "content": classify_prompt.format(
            topic="LLM 大语言模型",
            keywords="LLM, reasoning, ...",
            papers_text=f"[0] {paper.title}\n"
                        f"{paper.summary[:200]}"
        )
    }],
    response_format={"type": "json_object"}
)

result = json.loads(
    resp.choices[0].message.content
)
print(result)
```

审计要点

和 L13/L14 完全一致——先看输出对不对：

1. 返回的是 **JSON** 吗？如果是纯文本说明 Prompt 需要调整
2. **score** 合理吗？相关论文应该 > 0.8
3. **reason** 有意义吗？理由应该具体，不是套话
4. 中文摘要质量如何？是否覆盖了三要素

🎯 做法和 L13 的 `.head(5)` 一模一样：先拿一条数据跑通全流程，确认无误后再批量化。

[解释]

单篇测试的意义

LLM 的输出具有不确定性。同样的 Prompt 可能在不同论文上产生不同质量的结果。单篇测试的目的是验证 Prompt 的基本有效性，并在发现问题时及时调整（例如：返回格式不是 JSON、评分标准不符合预期、中文摘要遗漏关键信息等）。

调试完成后再进入批量处理，可以避免在大量 API 调用后才发现 Prompt 有问题，造成时间和费用浪费。

批量处理：循环 + 错误处理

核心代码

```
import time

results = []
for i, paper in enumerate(feed.entries[:20]):
    print(f"处理 {i+1}/20: {paper.title[:40]}...")

    try:
        # 调用 LLM 分类 + 摘要
        result = process_paper(paper)
        results.append(result)
    except Exception as e:
        print(f"⚠️ 失败: {e}")
        continue

    # 礼貌延迟
    time.sleep(1)

print(f"\n✅ 成功处理 {len(results)}/20 篇")
```

工程化要点

1. try/except 错误处理

单篇失败不影响整体流水线

2. time.sleep(1) 频率控制

避免触发 API 限流（429 错误）

3. 进度打印

`print(f"处理 {i+1}/20")` 让你知道进度

4. 部分成功即可用

20 篇中处理成功 18 篇，结果依然有价值

[解释]

批量处理的错误处理策略

LLM API 调用可能因为多种原因失败：网络超时、返回格式异常、API 限流（HTTP 429）等。用 try-except 包裹每次调用，记录失败原因但继续处理下一篇，是生产环境中的标准做法。

Alia-Agent 的 `classifyArticles()` 函数还实现了更复杂的重试机制（最多重试 2 次，失败则跳过），但教学版简化为“失败即跳过”即可。

结构化输出：JSON → DataFrame

合并结果

```
import pandas as pd

df = pd.DataFrame(results)

# 按相关性评分排序
df = df.sort_values("score", ascending=False)

# 选择关键列
df = df[["title_zh", "score", "reason",
        "summary_zh", "link"]]

# 保存
df.to_csv("arxiv_reading_list.csv",
          index=False)
print(f"✅ 已生成速读清单: {len(df)} 篇")
```

输出示例

中文标题	评分	摘要
面向长文本的高效注意力机制	0.92	本文提出 Flash-Attn 的改进版...
多模态 RAG 框架	0.85	结合视觉和文本检索的新方法...
基于强化学习的代码生成	0.78	使用 RLHF 训练代码模型...

🎯 最终产出就是这张表——中文标题、评分、摘要、原文链接。教师可以快速扫一遍，挑出值得深读的论文。

[解释]

从 JSON 到 DataFrame

每篇论文经过 LLM 处理后得到一个 JSON 对象（包含 score、reason、title_zh、summary_zh 等字段）。将所有 JSON 对象收集到列表中，用 `pd.DataFrame(results)` 一行代码即可转化为表格。

这个过程体现了 L15 的核心教学点：**LLM 的作用是把非结构化文本（英文摘要）转化为结构化数据（评分+中文摘要），从而让数据重新进入 L13/L14 所教授的分析流水线。**

成本核算：Token 用量与费用

Token 是什么？

- LLM 的计费单位
- 1 个中文字 ≈ 1-2 个 Token
- 1 个英文单词 ≈ 1 个 Token

AIIA-Agent 的真实生产数据

教师的 AIIA-Agent 系统每天运行 4 个主题的论文推送，以下是实际消耗：

指标	实测值
主题数	4 个（LLM / VLM / RAG / AI 应用）
每主题处理量	全量分类 + Top 20 篇摘要
日均 Token 消耗	124.7K Tokens
日均费用	¥0.28
月均费用	约 ¥8.5

费用拆解

模型：DeepSeek-V3.2（硅基流动）

环节	说明
分类	4 个主题 × 全量论文（约 200-400 篇/天）
摘要	4 × 20 = 80 篇精选论文的中文翻译
总 Token	124.7K / 天
总费用	¥0.28 / 天

对比参照

- 手动读完等量论文：8-12 小时
- AI 自动处理：5-10 分钟 + ¥0.28

💡 每天不到三毛钱，换来 8 小时的阅读时间。这就是 LLM API 的工程价值。

[解释]

Token 与成本控制

Token 是 LLM 处理文本的计量单位。大多数 LLM 按输入 Token 和输出 Token 分别计费，输出 Token 通常比输入贵 2-4 倍。

以上数据来自 AIIA-Agent 的生产环境实测（模型 DeepSeek-V3.2，平台硅基流动），涵盖 4 个主题（LLM、VLM、RAG、AI 应用）的完整流水线。控制成本的关键方法：1) 精简 Prompt（减少输入 Token）；2) 限制输出长度（100-150 字的约束）；3) 先用分类过滤低相关论文，只对高分论文生成摘要（从几百篇缩减到 80 篇）；4) 对重复出现的论文使用 PostgreSQL 缓存（避免二次调用 LLM）。

五、总结与展望

从"个人助手"到"生产级系统"

[解释]

第五章：总结与展望

本章回顾完整链路，展示 AIIA-Agent 的生产级实现作为对照，总结核心守则，并预告下一讲的收官项目。
本章包含 4 页内容：完整回路复盘、工业级对照、核心守则、课堂收获。

完整回路：从 RSS 到速读清单



本讲新技能

- ☒ RSS API 获取数据 (feedparser)
- ☒ LLM API 调用 (OpenAI SDK)
- ☒ Prompt 工程 (分类 + 摘要)
- ☒ JSON 结构化输出解析
- ☒ 批量处理 + 错误处理

与前两讲的共通原则

- 先打样再量产 (L13 `.head(5)` → L15 单篇测试)
- 批量化扩展 (L14 单源 → 多源 → L15 单篇 → N篇)
- 频率控制 (L14 `time.sleep(2)` → L15 同理)
- 代码审计 (L13 逻辑审计 → L15 输出审计)

[解释]

三讲的统一框架

L13-L15 的教学结构具有统一性：每讲都遵循“获取数据→处理数据→输出结果”的三段式，区别仅在于数据类型和处理工具。这种一致性是有意为之的——让学生意识到，不管数据是数字、网页还是文本，工程流程和协同模式是通用的。

工业级对照：AIIA-Agent 的论文推送系统

教学版（今天做的）

- 语言：Python
- 运行方式：Jupyter Notebook 手动执行
- 输出：CSV 文件
- 缓存：无
- 部署：本地运行

能力边界

适合个人使用：偶尔手动跑一次，获取最新论文列表

生产版（AIIA-Agent）

- 语言：Node.js
- 运行方式：服务器 Cron 定时任务，每天自动执行
- 输出：HTML 页面 + 企微群自动推送 + 微信公众号自动发布
- 缓存：PostgreSQL 数据库（避免重复调用 LLM）
- 部署：Docker 容器化

额外能力

- 引用数查询（Semantic Scholar API）
- HuggingFace 热度数据
- 综合评分公式： $0.5 \times \text{相关性} + 0.2 \times \text{引用} + 0.3 \times \text{热度}$

[解释]

教学版 vs 生产版

教学版的目标是让学生理解“LLM 处理非结构化文本”的基本原理和代码实现。生产版（AIIA-Agent）在此基础上增加了大量工程化组件：

1. **PostgreSQL 缓存**（db-arxiv.js）：已分类/翻译的论文写入数据库，下次遇到同一篇时直接读取，节省 LLM 调用成本
2. **热度评估**：结合 Semantic Scholar 引用数和 HuggingFace Daily Papers 点赞数，计算综合排名
3. **多通道分发**：自动格式化为 Markdown 消息和 HTML 页面，推送到企微信群聊，同时通过 RPA(机器人流程自动化) 技术自动发布到微信公众号
这些都是“课程之外”的工程实践，但了解它们有助于理解“从原型到产品”的距离。

核心守则

API Key 安全

-  严禁在代码中硬编码 Key
-  严禁截图发群、发朋友圈
-  使用环境变量 `os.getenv()`
-  如果泄露，立即在平台上重新生成

LLM 输出审核

- LLM 可能"幻觉"：生成不存在的信息
- 必须抽检 5-10% 的结果
- 摘要是否准确反映了原文
- 分类评分是否合理

成本控制

- 先小样本测试，再批量化
- 使用轻量模型处理简单任务
- 缓存已处理的结果

学术伦理

- 引用论文时注明 arXiv 来源
- AI 生成的摘要标注"AI 辅助翻译"
- LLM 的分类仅供参考，不能替代学术判断

[解释]

LLM 幻觉与审计

"幻觉" (Hallucination) 是 LLM 的已知问题：模型可能生成看起来合理但实际不正确的内容。在论文摘要场景中，可能表现为：编造不存在的方法名称、错误归纳论文结论、混淆相似论文的内容等。

防御策略：1) 抽检 5-10% 的摘要，与原文对照；2) 对评分异常的论文（如摘要明显与主题无关但评分很高）进行人工复核；3) 使用多个模型交叉验证关键结果。

课堂收获与下一讲预告

核心收获

- **✓ RSS API**: 通过官方接口获取数据
- **✓ LLM API 调用**: OpenAI SDK + 硅基流动
- **✓ Prompt 工程**: 分类 Prompt + 摘要 Prompt
- **✓ JSON 解析**: LLM 输出 → DataFrame
- **✓ 批量化工程**: 循环 + 错误处理 + 频率控制
- **✓ 安全意识**: API Key 保护 + 输出审核

下一讲预告

第十六讲：模块收官项目

综合运用 L13-L15 全部技能：

- 数据采集（爬虫 / API）
- 数据清洗（EDA + 多源对齐）
- 文本分析（LLM 分类 + 摘要）

完成一个从零到一的完整数据工程项目。

[解释]

模块四知识体系回顾

L13-L15 构成了数据工程模块的完整技能栈：

- **L13**: 结构化数据处理（EDA、清洗、统计分析）
- **L14**: 数据采集（HTML 爬虫、多源对齐、SSOT 原则）
- **L15**: 非结构化文本处理（RSS API、LLM 分类与摘要、JSON 解析）

L16 的收官项目将要求学生综合运用这三讲的能力，独立完成一个包含"采集→清洗→分析→LLM 处理→结论"的完整链路。